UNIVERSIDAD PRIVADA DEL NORTE
FACULTAD DE INGENIERÍA
Ingeniería de Sistemas Computacionales

PROGRAMACIÓN DINÁMICA APLICADA A LA EMPRESA SMARTOPTIMIZER S.A.C.

Práctica de Campo – Semana 5

Curso: Análisis de Algoritmos y Estrategias de Programación
Docente: Herber Edinson De La Cruz Pampañaupa
Grupo: N° 7

Integrantes

Apellidos y nombres	Código	Participación
BENITES VILLAVICENCIO, Paolo Sergio	N00XXXXXX	100 %
CANO ROJAS, Mijael Deyvi	N00210770	100 %
CASSINA MUÑOZ, Andres Guillermo	N00XXXXXX	100 %
QUISPE HUAMANI, Nayda	N00XXXXXX	100 %
ZUTTA LOPEZ, Deyvith Anderson	N00455989	100 %

Lima – Perú
2026

ÍNDICE

1. INTRODUCCIÓN 3
1.1 Contexto de la empresa 3
1.2 Problemática 3
1.3 Objetivos 3
1.4 Fundamento teórico 3
1.5 Estructura de la aplicación 3
2. DESARROLLO DE LAS CASUÍSTICAS 4
2.1 Casuística 01: Optimización de rutas de desplazamiento 4
2.2 Casuística 02: Optimización del ingreso de técnicos 7
2.3 Casuística 03: Selección óptima de proyectos tecnológicos 10
3. CONCLUSIONES 13
4. RECOMENDACIONES 13
REFERENCIAS 13
ANEXOS 13
Anexo A. Repositorio del proyecto 13

1. INTRODUCCIÓN

1.1 Contexto de la empresa

SmartOptimizer S.A.C. se dedica al desarrollo de soluciones tecnológicas para empresas. La empresa identificó que muchos de sus clientes toman decisiones con procedimientos manuales, en los que se revisa una gran cantidad de alternativas antes de elegir una. Por eso pidió un sistema en Python, con interfaz gráfica en Tkinter, que permita elegir un módulo de optimización, ingresar los datos y ver la solución obtenida.

1.2 Problemática

Los tres casos planteados por la empresa (rutas de desplazamiento de los técnicos, niveles de certificación y selección de proyectos) tienen en común que el número de alternativas crece muy rápido cuando aumentan los datos. Revisarlas todas a mano o con fuerza bruta no es práctico, y muchas de esas alternativas repiten los mismos cálculos parciales. Esta situación es la que resuelve la Programación Dinámica: cada subproblema se calcula una vez, se guarda en una tabla y se reutiliza (Cormen et al., 2022).

1.3 Objetivos

Objetivo general

Desarrollar en Python una aplicación con interfaz Tkinter que aplique Programación Dinámica para resolver las tres casuísticas de SmartOptimizer S.A.C. y que muestre la tabla de resultados y el análisis de cada solución.

Objetivos específicos

1. Calcular el costo mínimo para que un técnico vaya de la posición 0 a la 10 y reconstruir la ruta.
2. Determinar cuántas formas existen de llegar al nivel 8 de certificación y mostrar los resultados parciales y la relación de recurrencia.
3. Seleccionar los proyectos que dan el mayor beneficio sin elegir dos consecutivos.

1.4 Fundamento teórico

La Programación Dinámica divide un problema en subproblemas más pequeños que se repiten y guarda la respuesta de cada uno para no volver a calcularla. Para aplicarla, el problema debe tener subestructura óptima, es decir, que la mejor solución se pueda formar con las mejores soluciones de sus subproblemas, y subproblemas superpuestos (Kleinberg & Tardos, 2006). En esta práctica se usó la tabulación (de abajo hacia arriba), que parte de los casos base y va llenando una tabla, porque permite mostrar la tabla completa en la interfaz.

1.5 Estructura de la aplicación

El código está en dos archivos: dp_logica.py, con las tres funciones de Programación Dinámica, y smartoptimizer_app.py, con la ventana Tkinter que tiene una pestaña por módulo. Se ejecuta con python src/smartoptimizer_app.py.

2. DESARROLLO DE LAS CASUÍSTICAS

2.1 Casuística 01: Optimización de rutas de desplazamiento

Planteamiento

Los técnicos deben ir desde la posición 0 hasta la 10 y en cada movimiento pueden avanzar 1, 2 o 3 posiciones, con un costo de S/ 10, S/ 18 y S/ 25 respectivamente. Se definió dp[i] como el costo mínimo para llegar a la posición i. Como a la posición i solo se llega desde i−1, i−2 o i−3, se toma la opción más barata de las tres:

$$dp[i] = \text{mín}(dp[i-1] + 10 ; dp[i-2] + 18 ; dp[i-3] + 25), \quad dp[0] = 0$$

Junto con dp[i] se guarda el paso que dio el mínimo. Con esa información se reconstruye la ruta desde el destino hacia atrás.

Implementación

```
def ruta_costo_minimo(inicio, destino, costos):
    """Costo mínimo para ir de `inicio` a `destino`.

    costos: dict {pasos: costo}, p. ej. {1: 10, 2: 18, 3: 25}
    Recurrencia: dp[i] = min(dp[i - p] + costo[p])  para cada paso p con i - p >= inicio
    Caso base:   dp[inicio] = 0
    Complejidad: O(n * k) en tiempo y O(n) en espacio (n = posiciones, k = tipos de
paso).
    Devuelve (costo_minimo, tabla_dp, decision, camino).
    """
    if destino < inicio:
        raise ValueError("El destino debe ser mayor o igual que la posición inicial.")
    n = destino - inicio
    INF = float("inf")
    dp = [INF] * (n + 1)          # dp[j] = costo mínimo para llegar a la posición
inicio + j
    decision = [None] * (n + 1)   # paso usado para llegar a j (para reconstruir el
camino)
    dp[0] = 0
    for j in range(1, n + 1):
        for pasos, costo in sorted(costos.items()):
            if j - pasos >= 0 and dp[j - pasos] + costo < dp[j]:
                dp[j] = dp[j - pasos] + costo
                decision[j] = pasos

    # Reconstrucción del camino desde el destino hacia atrás
    camino, j = [], n
    while j > 0:
        p = decision[j]
        camino.append((inicio + j - p, inicio + j, p, costos[p]))
        j -= p
    camino.reverse()
    return dp[n], dp, decision, camino
```

Resultados

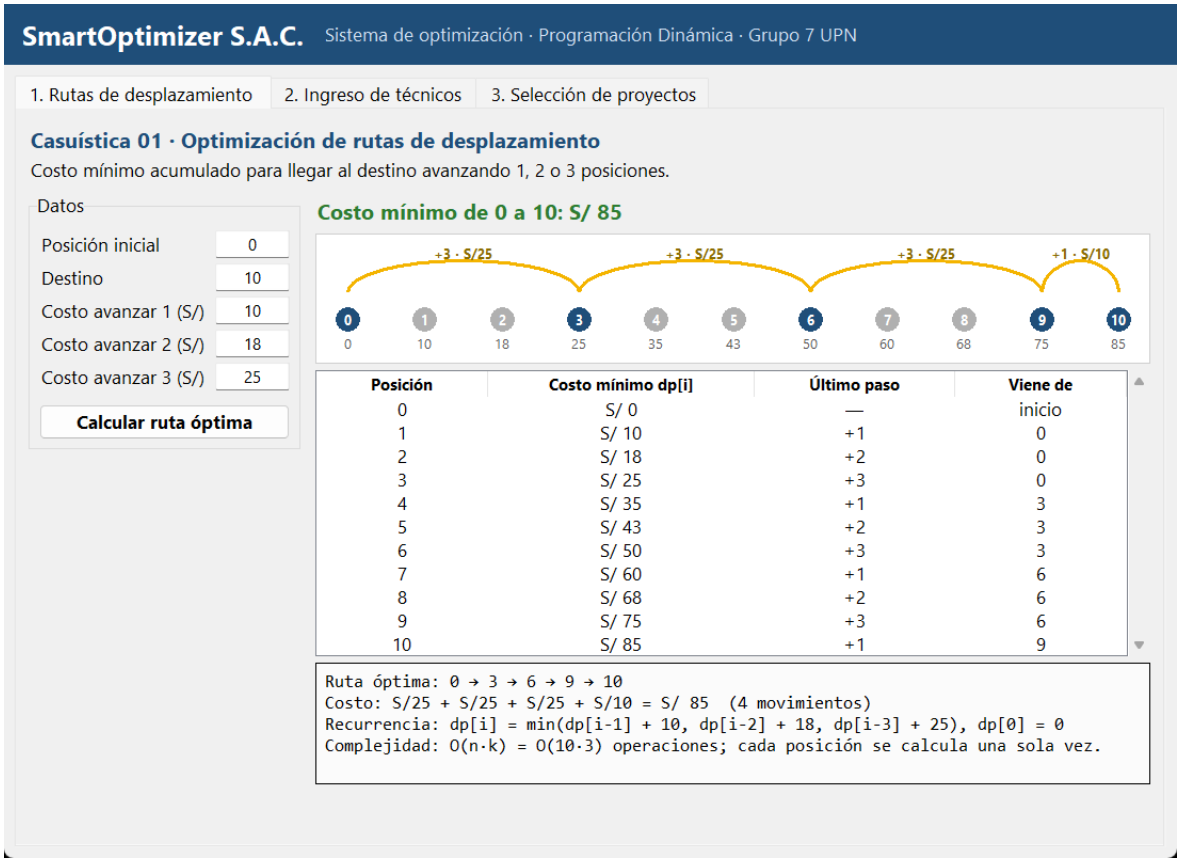
Tabla 1
Costo mínimo acumulado por posición

Posición	Costo mínimo (S/)	Último paso	Viene de
0	0	—	Inicio
1	10	+1	0
2	18	+2	0
3	25	+3	0
4	35	+1	3
5	43	+2	3
6	50	+3	3
7	60	+1	6
8	68	+2	6
9	75	+3	6
10	85	+1	9

Nota. Elaboración propia.

El costo mínimo para llegar de la posición 0 a la 10 es de S/ 85. La ruta pasa por las posiciones 0, 3, 6, 9, 10: tres avances de 3 posiciones (S/ 25 cada uno) y un último avance de 1 posición (S/ 10). El avance de 3 posiciones es el que conviene usar más, porque cuesta S/ 8,33 por posición, mientras que el de 2 cuesta S/ 9 y el de 1, S/ 10. En algunas posiciones hay empate; por ejemplo, a la posición 4 se llega con S/ 35 por dos caminos distintos. En esos casos la tabla guarda solo uno, ya que el costo es el mismo.

Figura 1
Ejecución del módulo 1: ruta óptima, tabla y relación de recurrencia



Nota. Elaboración propia.

Análisis de complejidad

Cada posición se calcula una sola vez revisando los 3 tipos de avance, por lo que el tiempo es O(n·k) y el espacio O(n). Para el caso pedido son 30 comparaciones. Si se probaran todas las secuencias de avances que suman 10 posiciones habría que revisar 274 combinaciones, y ese número crece de forma exponencial con el destino.

2.2 Casuística 02: Optimización del ingreso de técnicos

Planteamiento

Los técnicos nuevos deben superar 8 niveles de certificación y pueden avanzar 1, 2 o 3 niveles a la vez. Se pide el número de formas distintas de llegar al nivel 8. Cualquier forma de llegar al nivel n termina con un avance de 1, 2 o 3 niveles, así que el total es la suma de las formas de llegar a los tres niveles anteriores:

$$f(n) = f(n-1) + f(n-2) + f(n-3), \quad f(0) = 1, \quad f(n) = 0 \text{ si } n < 0$$

El caso base f(0) = 1 representa la única forma de estar en el nivel inicial, que es no haber avanzado. Esta relación de recurrencia es la de la sucesión de Tribonacci.

Implementación

```
def formas_de_llegar(niveles, saltos=(1, 2, 3)):
    """Número de formas distintas de llegar al nivel `niveles` avanzando con `saltos`.

    Recurrencia: f(n) = f(n-1) + f(n-2) + f(n-3)   (con saltos 1, 2 y 3)
    Caso base:   f(0) = 1 (una forma de "no moverse"); f(n) = 0 si n < 0
    Complejidad: O(n * k) en tiempo y O(n) en espacio.
    Devuelve (total, tabla_f, detalle) donde detalle[i] es el texto del cálculo de f(i).
    """
    if niveles < 0:
        raise ValueError("La cantidad de niveles no puede ser negativa.")
    f = [0] * (niveles + 1)
    f[0] = 1
    detalle = ["f(0) = 1  (caso base)"]
    for i in range(1, niveles + 1):
        partes, nombres = [], []
        for s in saltos:
            if i - s >= 0:
                f[i] += f[i - s]
                partes.append(str(f[i - s]))
                nombres.append(f"f({i - s})")
        detalle.append(f"f({i}) = {' + '.join(nombres)} = {' + '.join(partes)} =
{f[i]}")
    return f[niveles], f, detalle
```

Resultados parciales

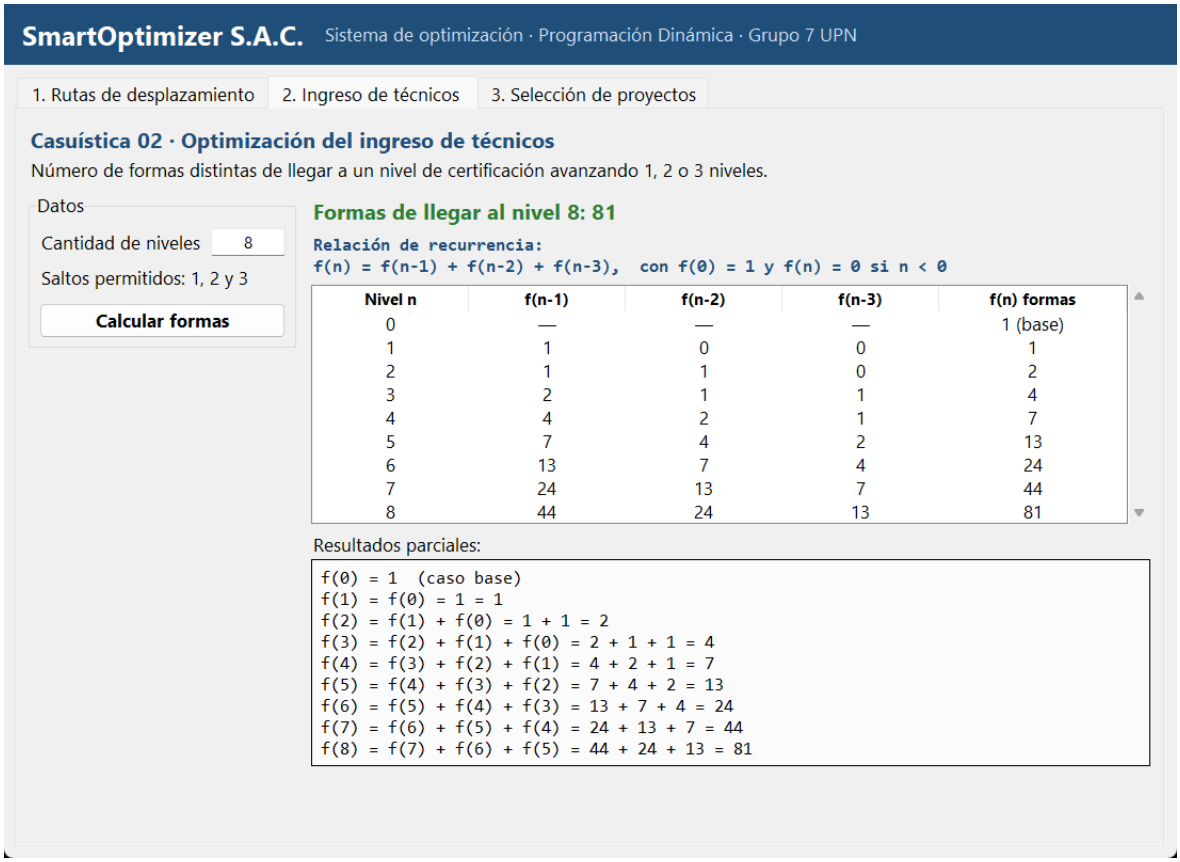
Tabla 2
Número de formas de llegar a cada nivel

Nivel n	f(n−1)	f(n−2)	f(n−3)	f(n)
0	—	—	—	1 (caso base)
1	1	0	0	1
2	1	1	0	2
3	2	1	1	4
4	4	2	1	7
5	7	4	2	13
6	13	7	4	24
7	24	13	7	44
8	44	24	13	81

Nota. Elaboración propia.

Existen 81 formas distintas de llegar al nivel 8. En la tabla se ve que cada valor se obtiene sumando los tres anteriores; por ejemplo, f(8) = 44 + 24 + 13 = 81. El número de formas casi se duplica de un nivel al siguiente, por lo que contarlas una por una se vuelve cada vez más costoso.

Figura 2
Ejecución del módulo 2: relación de recurrencia y resultados parciales



Nota. Elaboración propia.

Análisis de complejidad

La tabla se llena con 3 sumas por nivel, así que el tiempo es O(n) y el espacio O(n). Como solo se usan los tres valores anteriores, el espacio podría bajar a O(1). Una solución recursiva sin guardar resultados repetiría los mismos cálculos y su costo sería cercano a $O(3^n)$.

2.3 Casuística 03: Selección óptima de proyectos tecnológicos

Planteamiento

SmartOptimizer tiene 8 proyectos con beneficios conocidos, pero no puede elegir dos proyectos consecutivos porque usan el mismo equipo especializado. Se definió dp[i] como el mayor beneficio posible usando los proyectos P1 a Pi. Para cada proyecto hay dos opciones: no elegirlo y mantener dp[i−1], o elegirlo, lo que obliga a dejar el anterior y sumar su beneficio a dp[i−2]:

$$dp[i] = máx(dp[i-1] ; dp[i-2] + b[i]), \quad dp[0] = 0, \quad dp[1] = b[1]$$

Implementación

```
def seleccion_proyectos(beneficios):
    """Máximo beneficio sin elegir dos proyectos consecutivos.

    Recurrencia: dp[i] = max(dp[i-1],  dp[i-2] + b[i])
                 (no tomar el proyecto i  |  tomarlo y saltar el anterior)
    Caso base:   dp[0] = 0, dp[1] = b[1]
    Complejidad: O(n) en tiempo y O(n) en espacio.
    Devuelve (maximo, dp, seleccionados, descartados, filas) con índices 1..n.
    """
    n = len(beneficios)
    b = [0] + list(beneficios)            # b[1..n]
    dp = [0] * (n + 1)
    if n >= 1:
        dp[1] = b[1]
    filas = []                             # para mostrar la tabla DP paso a paso
    if n >= 1:
        filas.append((1, b[1], 0, b[1], dp[1], "Tomar"))
    for i in range(2, n + 1):
        sin_i = dp[i - 1]
        con_i = dp[i - 2] + b[i]
        dp[i] = max(sin_i, con_i)
        filas.append((i, b[i], sin_i, con_i, dp[i], "Tomar" if con_i > sin_i else
"Saltar"))

    # Reconstrucción: si dp[i] viene de tomar i, se elige i y se salta a i-2
    seleccionados, i = [], n
    while i >= 1:
        if i == 1 or dp[i - 2] + b[i] > dp[i - 1]:
            if i == 1 and dp[1] == 0:
                break
            seleccionados.append(i)
            i -= 2
        else:
            i -= 1
    seleccionados.reverse()
    descartados = [i for i in range(1, n + 1) if i not in seleccionados]
    return dp[n], dp, seleccionados, descartados, filas
```

Resultados

Tabla 3
Tabla de Programación Dinámica para la selección de proyectos (S/)

Proyecto	Beneficio	Sin Pi: dp[i−1]	Con Pi: dp[i−2] + b[i]	dp[i]	Decisión	Elegido
P1	4,000	0	4,000	4,000	Tomar	Sí
P2	2,500	4,000	2,500	4,000	Saltar	No
P3	5,000	4,000	9,000	9,000	Tomar	Sí
P4	3,500	9,000	7,500	9,000	Saltar	No
P5	6,000	9,000	15,000	15,000	Tomar	Sí
P6	4,500	15,000	13,500	15,000	Saltar	No
P7	7,000	15,000	22,000	22,000	Tomar	Sí
P8	3,000	22,000	18,000	22,000	Saltar	No

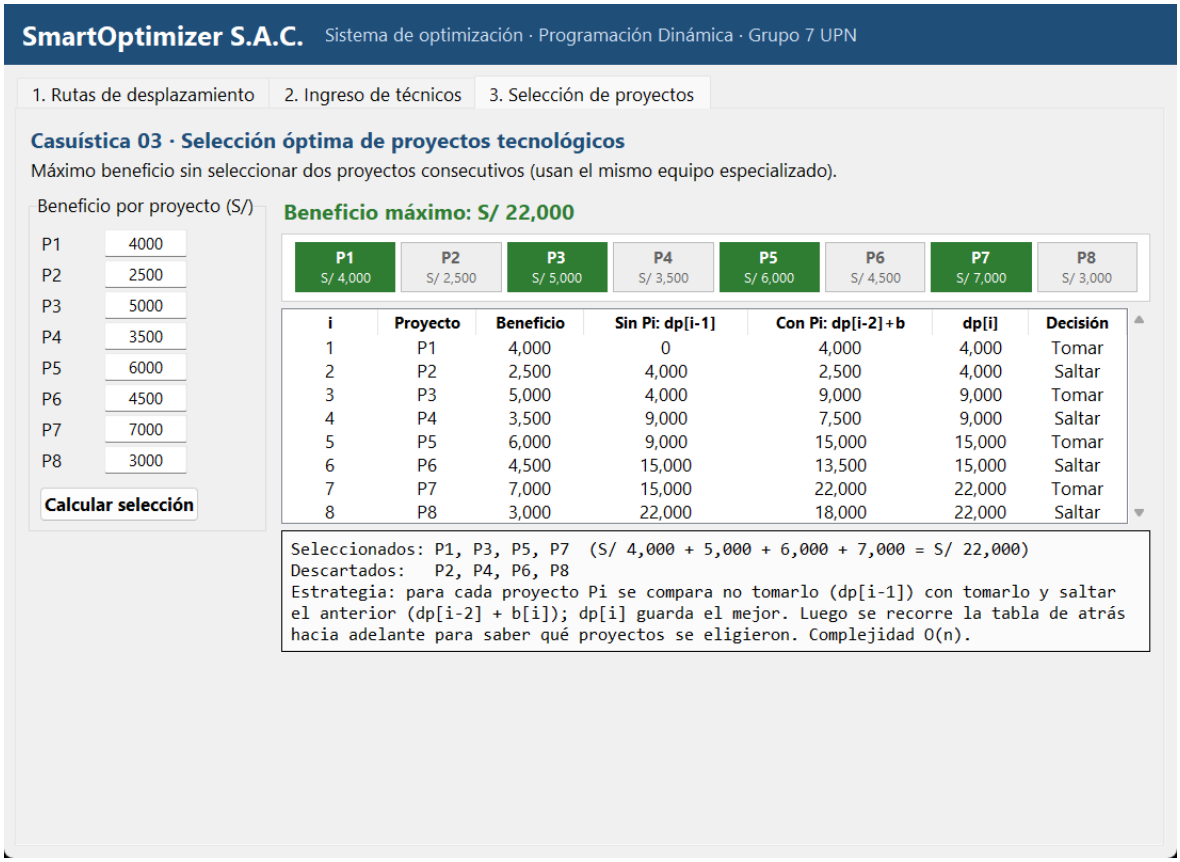
Nota. Elaboración propia.

El beneficio máximo es de S/ 22,000, con los proyectos P1, P3, P5 y P7 (S/ 4,000 + 5,000 + 6,000 + 7,000). Se descartan P2, P4, P6 y P8. Entre los proyectos elegidos no hay dos consecutivos, por lo que se cumple la restricción del equipo especializado.

La estrategia fue llenar la tabla de P1 a P8 comparando en cada paso las dos opciones y luego recorrerla desde el final. Si dp[i] se obtuvo eligiendo Pi, el proyecto se marca y se pasa a i−2; si no, se pasa a i−1. Por ejemplo, en P8 conviene no elegirlo: con él se obtendrían S/ 18,000 (dp[6] + 3,000), menos que los S/ 22,000 que ya se tenían con P7.

Figura 3

Ejecución del módulo 3: proyectos elegidos, tabla y estrategia



Nota. Elaboración propia.

Análisis de complejidad

Cada proyecto se evalúa una sola vez con una comparación, por lo que el tiempo es O(n) y el espacio O(n). Revisar todas las combinaciones posibles costaría $O(2^n)$: para 8 proyectos son 256 subconjuntos, y con unas decenas de proyectos ya no sería viable.

3. CONCLUSIONES

1. La aplicación resuelve los tres casos planteados: la ruta de la posición 0 a la 10 tiene un costo mínimo de S/ 85, existen 81 formas de llegar al nivel 8 y el mayor beneficio sin proyectos consecutivos es de S/ 22,000, con P1, P3, P5 y P7.
2. En los tres módulos se siguió el mismo procedimiento: definir qué representa cada valor de la tabla, plantear la relación de recurrencia, fijar los casos base y llenar la tabla desde el caso más pequeño.
3. Con Programación Dinámica, problemas que por fuerza bruta tienen un costo exponencial se resuelven en tiempo lineal, porque cada subproblema se calcula una sola vez.
4. Guardar la decisión tomada en cada paso permitió obtener no solo el valor óptimo, sino también la ruta y la lista de proyectos elegidos.

4. RECOMENDACIONES

1. Si la empresa trabaja con más niveles o proyectos, conviene guardar solo los últimos valores de la tabla para reducir el uso de memoria.
2. Los costos de desplazamiento podrían leerse desde un archivo de configuración, para no tener que ingresarlos cada vez que cambian.

REFERENCIAS

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson Education.

Python Software Foundation. (2024). *tkinter: Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

ANEXOS

Anexo A. Repositorio del proyecto

El código fuente completo y las capturas de ejecución se encuentran en: https://github.com/MijaelCAN/analisis-algoritmos-upn-2026/tree/main/PC5-Programacion-dinamica